



Generative AI inference architecture and best practices on AWS

AWS Prescriptive Guidance



AWS Prescriptive Guidance: Generative AI inference architecture and best practices on AWS

Copyright © 2026 Amazon Web Services, Inc. and/or its affiliates. All rights reserved.

Amazon's trademarks and trade dress may not be used in connection with any product or service that is not Amazon's, in any manner that is likely to cause confusion among customers, or in any manner that disparages or discredits Amazon. All other trademarks not owned by Amazon are the property of their respective owners, who may or may not be affiliated with, connected to, or sponsored by Amazon.

Table of Contents

Introduction	1
Intended audience	1
Objectives	2
AWS inference stack	3
Serverless inference layer	3
Managed inference layer	4
Self-managed inference layer	4
Components of an AI inference stack	5
End user application	5
Inference API server or frontend	6
Inference backend	6
Challenges of inference compared to training	7
Selecting an AWS service for AI inference	9
Amazon Bedrock	9
Infrastructure management	9
Pricing model	9
Model architecture support	10
Automatic scaling	10
Inference engine choice	10
Inference configuration	10
Supported clients and protocols	10
Amazon SageMaker AI inference	10
Infrastructure management	10
Pricing model	11
Model architecture support	11
Automatic scaling	11
Inference engine choice	11
Inference configuration	11
Supported clients and protocols	12
Amazon SageMaker HyperPod	12
Infrastructure management	12
Training compute reuse	12
Pricing model	12
Model architecture support	12

Automatic scaling	13
Inference engine choice	13
Inference configuration	13
Supported clients and protocols	13
Self-managed inference	14
Infrastructure management	14
Training compute reuse	14
Pricing model	14
Model architecture support	14
Automatic scaling	14
Inference engine choice	15
Inference configuration	15
Supported clients and protocols	15
Model optimization techniques	16
Pruning	16
Quantization	16
Model compilation	16
Speculative decoding	16
Artifact storage	17
Applying inference optimizations in SageMaker AI	17
Right-sizing and auto-scaling an inference system	18
Selecting Amazon EC2 instance types and instance count	18
Configuring auto-scaling policies	20
System-level optimizations	22
Implementing security and responsible AI	23
Before moving to inference	23
Model assessment	23
Documentation preparation	24
Technical safeguards	24
Governance setup	24
After inference deployment	25
Continuous monitoring	26
Incident response mechanisms	26
Feedback collection mechanisms	26
AWS Partner Network	28
Resources	29

AWS service documentation	29
Other AWS resources	29
Other resources	29
Contributors	30
Document history	31

Generative AI inference architecture and best practices on AWS

Oussama Kandakji, Mia Chang, Vivek Gangasani, Paco Gonzalez, Nicolas Jourdan, Christian Kamwangala, Marco Sgroi, and Hamza Usmani, Amazon Web Services

Organizations that deploy generative AI (gen AI) models such as large language models (LLMs) and multimodal generation models face critical challenges in serving production workloads at scale. These challenges span multiple dimensions including how to:

- Maintain consistent low-latency responses for real-time applications and user experiences.
- Implement dynamic capacity scaling to handle unpredictable traffic patterns.
- Optimize infrastructure costs.
- Ensure high availability across deployments.

This guide provides comprehensive prescriptive guidance on selecting and implementing appropriate AWS inference services, designing resilient architectures, and applying proven best practices. This guidance can help organizations achieve performant, cost-effective, and reliable gen AI deployments

Intended audience

This guide is intended for the following:

- AI and machine learning (ML) teams who have trained gen AI models, like [LLMs](#) and multimodal generation models such as [diffusion models](#), and want to deploy them for production inference
- Solution architects and technical leaders evaluating AWS inference options.
- Organizations transitioning from model development to production deployment.
- AWS Partners and system integrators providing inference solutions and implementation services.
- Technical decision-makers assessing infrastructure requirements for AI model inference.

To understand the concepts and recommendations in this guide, you should have a basic understanding of ML concepts and AWS services.

Objectives

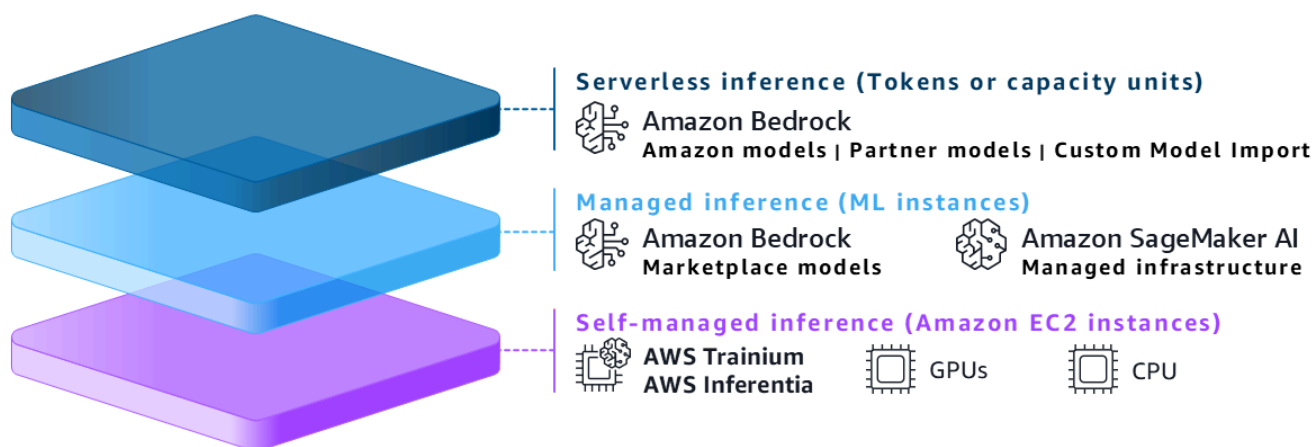
The recommendations in this guide can help you achieve the following:

- Understand the differences between AI training and inference workloads and their unique challenges.
- Navigate the AWS inference stack (serverless, managed, and self-managed) and select the appropriate service based on key decision criteria.
- Learn about model and system optimization techniques for efficient inference deployment.
- Review recommended security controls and learn about responsible AI (RAI) practices for production inference workloads.
- Leverage the AWS Partner Network to accelerate time-to-market and model distribution.

AWS inference stack

AI inference is the process of using a trained machine learning (ML) model to make predictions based on input data. Traditional ML inference typically involves relatively small models with modest compute requirements. However, generative AI (gen AI) inference has introduced new requirements in terms of compute, specialized hardware, and system optimizations.

As shown in the following diagram, AWS provides a multi-layer AI inference stack to address customers' diverse inference needs.



2

Following are descriptions of each layer of the AI inference stack.

Serverless inference layer

Following are key capabilities of the serverless inference layer, which is the top layer of the AI inference stack:

- Powered by [Amazon Bedrock](#) which offers a wide choice of industry leading foundation models (FMs) along with a broad set of capabilities to build generative AI applications.

- Supports importing custom models with popular architectures such as Llama, Mistral, and Qwen.
- Abstracts infrastructure management completely including high availability, scaling, and performance tuning.
- Pay-as-you-go pricing by using tokens or capacity units. For more information, see [Amazon Bedrock pricing](#).
- Ideal for organizations who want to minimize operational and development effort.

Managed inference layer

Following are key capabilities of the managed inference layer, which is the middle layer of the AI inference stack:

- Powered by [Amazon SageMaker AI](#) which provides a broad set of tools to enable high-performance, low-cost ML for building, training and deploying AI models at scale. It provides all these capabilities in one integrated development environment (IDE).
- Enables serving pre-trained or custom models on managed infrastructure.
- Balances control with simplicity and with flexibility in choosing inference engines, scaling policies, and instance types.
- Suitable for organizations who want more control over deployment configuration, scaling behavior, and cost optimization while maintaining operational simplicity.

Self-managed inference layer

Following are key capabilities of the self-managed inference layer, which is the bottom layer of the AI inference stack:

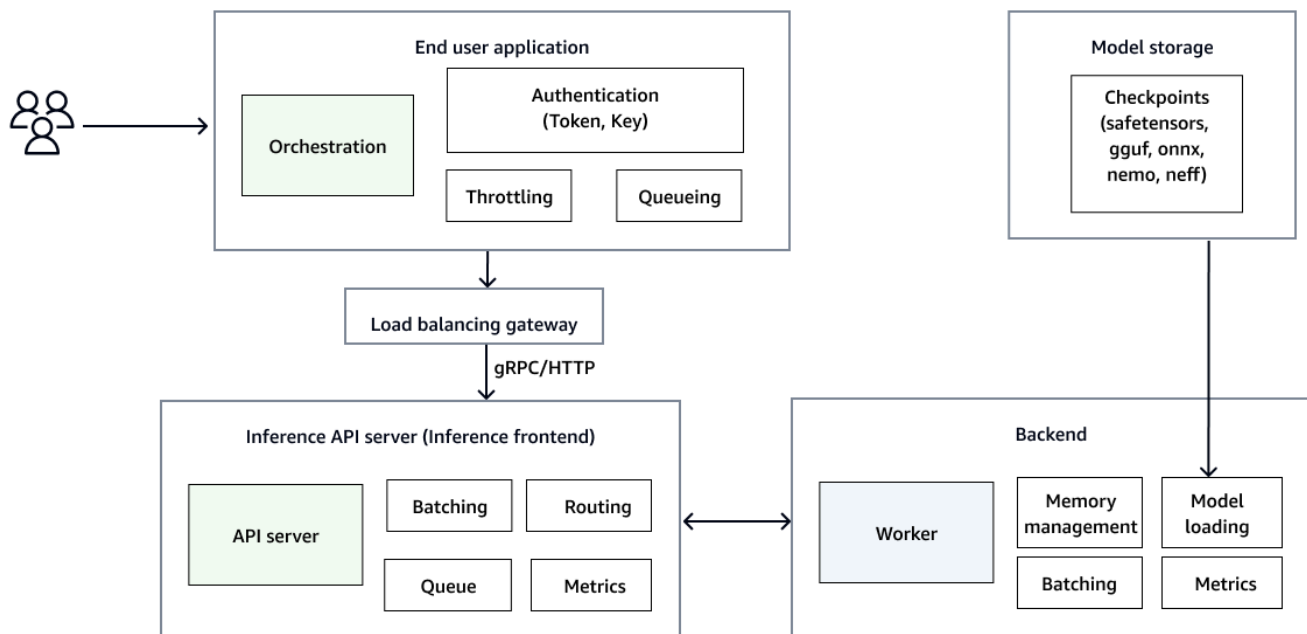
- Powered by [Amazon EC2 instances](#) with [AWS Trainium](#) and [AWS Inferentia](#) chips, AWS and NVIDIA [GPUs](#), and CPUs. Offers broad and deep compute capabilities with over 1,000 instances and choice of the latest processor, storage, networking, operating system, and purchase model.
- Can operate through services like [Amazon Elastic Kubernetes Service](#) (Amazon EKS) or [Amazon Elastic Container Service](#) (Amazon ECS).
- Offers maximum flexibility and control of their infrastructure and software.
- Ideal for organizations who require complete control over their infrastructure.

Components of an AI inference stack

A production AI inference stack typically consists of the following three core components:

- End user application – Serves as the entry point for user interactions and request handling
- Inference API server (inference frontend) – Manages API serving and request processing
- Inference backend – Handles the core model execution and resource management

These components communicate through standardized protocols like [gRPC](#) for reliable and efficient model serving. The following diagram shows the high-level architecture of a real-time inference workload. It showcases the core components and how they communicate with each other.



This section provides details about each component.

End user application

The end user application component serves as the initial touchpoint for all client interactions, providing a secure and controlled entry point to the inference system. It incorporates essential security features like authentication through tokens and keys, implements throttling mechanisms to prevent system overload, and includes queuing capabilities to manage high request volumes. Some emerging tools use proxies and load balancers to enable key provisioning, multi-tenant or

multi-user throttling, queueing, and automatic failover in multi-region scenarios. Examples of these emerging tools include [LiteLLM](#), [Portkey](#), and [Kong AI Gateway](#).

Inference API server or frontend

Acting as the bridge between client requests and model execution, the frontend component provides a robust API server that orchestrates the flow of inference requests. This component implements sophisticated request handling mechanisms including the following:

- Request batching for improved throughput
- Intelligent routing to distribute load across available workers
- Queue management for optimal request processing
- Comprehensive metrics collection to monitor system performance

The frontend communicates with the backend through standardized gRPC protocols, ensuring reliable and efficient data transfer. Popular frontends include [NVIDIA Triton](#), [NVIDIA Dynamo](#), [Deep Java Library \(DJL\)](#), [Ray Serve](#), and [Text Generation Inference \(TGI\)](#) from Hugging Face.

Inference backend

The inference backend is where the model execution takes place, managed by specialized workers that handle the computational heavy lifting. The backend component is responsible for critical functions such as the following:

- Memory management to optimize resource utilization
- Model loading and initialization to ensure efficient startup
- Computation-level batching for improved throughput
- Performance metrics tracking

It works in close coordination with the frontend component to process requests while managing system resources effectively to maintain optimal performance and reliability. You can use a Python backend where you write your own inference logic. You can also use more optimized and popular backends such as [vLLM](#), [TensorRT LLM](#), [ONNX Runtime](#), and [SGLang](#). The backend component also maintains a connection to model storage, which houses various model formats including [Safetensors](#), [GGUF](#), [ONNX](#), [NeMo](#), [NEFF](#), and Hugging Face models. This capability helps provide efficient access to the required model checkpoints.

Challenges of inference compared to training

Training large foundation models is often seen as the most resource-intensive phase of the AI lifecycle. However, moving to inference brings a different set of technical and operational challenges.

Training workloads are typically predictable, compute-bound, and throughput-oriented, whereas inference workloads are often more unpredictable, memory-bound, and latency sensitive.

This shift impacts every aspect of system design—from scaling and resource management to operational security. The following table outlines the key differences between training and inference challenges.

Category	Training	Inference
Operational complexity	• Planned, fixed-duration runs with predictable resource needs. • Scaling needs are clear in advance.	• Continuous or bursty workloads for real-time inference. • Autoscaling and traffic management and routing are essential to handle dynamic demand.
Latency and throughput	• Optimized for throughput. • Latency is secondary	• Latency-sensitive and concurrency-heavy. • Must deliver low time-to-first-token while supporting many simultaneous sessions.
Resource management	• Primarily compute-bound. • Large models stay loaded throughout the training. • Checkpoint writing must be handled efficiently.	• Often memory-bound because of model size and per-request state. • Requires memory-efficient serving techniques to prevent compute underutilization.

	• Node failure management for long-running training workloads.	• Checkpoint loading must be efficient to enable fast scaling with dynamic demand.
Optimization strategies	• Focus on maximizing accelerator utilization and fast convergence • Optimizing data pipelines, using mixed precision. • Adaptive hyperparameters.	• Focus on minimizing per-request latency. • High efficiency and cost-per-request. • Leverage hardware accelerator architecture-aware optimizations, model compression, and caching to serve responses faster.
Security and compliance	• Controlled, internal environments with limited external exposure.	• Public or customer-facing endpoints. • Often requires tenant isolation and API key management. • Traffic encryption and compliance enforcement.
Cost control	• Predictable, capped costs tied to the length of the training run and the provisioned hardware resources	• Variable, usage-driven costs. • Risk of over-provisioning or idle spend because of request spikes. • Scaling must be carefully tuned to match demand patterns.

Selecting an AWS service for AI inference

When selecting an AWS service for AI inference, organizations need to consider multiple factors that align with their operational requirements and technical capabilities.

Key decision criteria to consider when choosing an AWS service include the following:

- Level of infrastructure management desired
- Existing compute commitments
- Pricing model preferences
- Required model architecture support
- Level of control needed in inference engine selection, configuration, and infrastructure

Relevant AWS services include Amazon Bedrock, Amazon SageMaker AI managed endpoints, Amazon SageMaker HyperPod, or self-managed infrastructure using Amazon EC2, Amazon EKS, and Amazon ECS.

Amazon Bedrock

[Amazon Bedrock](#) provides a fully managed environment for model inference, automatically handling all infrastructure provisioning and scaling. It is optimized for seamless use of off-the-shelf foundation models, while also supporting the import of customized models for select architectures.

Infrastructure management

AWS manages the complete infrastructure lifecycle, including provisioning, scaling, and maintenance. This capability allows teams to focus on application development.

Pricing model

Amazon Bedrock offers flexible pricing based on actual consumption. Organizations pay for input and output tokens processed during on-demand usage, or they can use and scale imported FMs measured in capacity units.

Model architecture support

Amazon Bedrock provides access to a wide selection of [foundation models](#), including Amazon Nova, Anthropic Claude, Mistral, Meta Llama, and Qwen. With the Custom Model Import feature, you can use custom models that align with [supported architectures](#), enabling deployment of fine-tuned models.

Automatic scaling

AWS automatically adjusts capacity based on demand patterns, which helps to provide consistent performance during traffic fluctuations.

Inference engine choice

Amazon Bedrock provides a fully managed inference engine optimized for the supported model architectures, with AWS handling all engine configuration and optimization.

Inference configuration

You can control model behavior through model-specific request [parameters](#). This capability allows customization of generation characteristics such as temperature, top-p sampling, and maximum token length for each inference request.

Supported clients and protocols

Applications can access models in Amazon Bedrock through the [InvokeModel](#) or [Converse](#) API actions over HTTPS or integrate using the [AWS SDK](#). This approach provides straightforward integration paths for various application architectures. You can also use third-party open source libraries like [Strands Agents](#) or [Crew AI](#) to interact with Amazon Bedrock.

Amazon SageMaker AI inference

SageMaker AI Inference endpoints deliver a managed inference environment specifically designed for deploying ML models at scale.

Infrastructure management

SageMaker AI manages the underlying ML instances while providing organizations with control over instance types and configurations. The service handles instance provisioning, health

monitoring, and infrastructure maintenance. With these capabilities, teams can focus on model deployment configuration with containers maintained by AWS or custom containers and create inference pipelines for multi-model workflows.

Pricing model

Customers are charged based on the instance types deployed, with costs covering both compute resources and storage. This instance-based [pricing model](#) enables accurate cost forecasting based on instance selection and deployment duration.

Model architecture support

SageMaker AI accommodates any compatible model framework or architecture, providing flexibility to deploy models built with TensorFlow, PyTorch, Hugging Face, scikit-learn, XGBoost, or custom frameworks.

Automatic scaling

You can configure automatic scaling policies that adjust the number of instances or replicas based on multiple factors. For example, factors include time schedules, built-in Amazon CloudWatch metrics such as invocation rates, concurrent requests, resource utilization, or custom metrics tailored to specific application requirements.

Inference engine choice

Teams can choose [AWS Deep Learning Containers](#) images that are optimized for popular frameworks and maintained by AWS. Or, teams can bring custom container images that include specialized inference engines or dependencies.

Inference configuration

The service enables flexible configuration of both model parameters and system-level settings, including the following:

- Model's precision, such as FP16, FP8, or INT4
- Engine-specific optimization strategies like prefix caching and speculative decoding to improve performance
- Model sharding techniques for distributing large models across multiple GPUs or Trainium [NeuronCores](#)

Supported clients and protocols

Applications can invoke endpoints through the Amazon SageMaker Runtime [API actions](#) over HTTPS, integrate using the AWS SDK, or leverage the [Amazon SageMaker Python SDK](#) for streamlined Python-based interactions.

Amazon SageMaker HyperPod

[Amazon SageMaker HyperPod](#) with Amazon EKS orchestration enables large-scale, distributed model inference with enterprise-grade orchestration capabilities. For more information, see [Orchestrating SageMaker HyperPod clusters with Amazon EKS](#) in the Amazon SageMaker AI documentation.

Infrastructure management

SageMaker HyperPod provides managed ML instances that AWS automatically provisions and monitors as part of the HyperPod cluster. Kubernetes orchestration through Amazon EKS enables sophisticated workload management and scheduling.

Training compute reuse

With SageMaker HyperPod, you can use the same compute resources for both training and inference workloads, maximizing infrastructure efficiency. Additionally, you can govern the usage of compute resources across different types of workloads by using [SageMakerHyperPod task governance](#).

Pricing model

Customers are charged based on provisioned instances covering compute and storage resources. [SageMaker training plans](#) are flexible and offer commitment-based pricing that provides reduced rates and guaranteed capacity for predictable workloads.

Model architecture support

The service accommodates any model format or architecture, with full customization of model frameworks and deployment configurations. This flexibility supports diverse use cases from standard transformer models to custom architectures with specialized requirements.

Automatic scaling

Pod scaling operates through [Kubernetes Event-driven Autoscaling \(KEDA\)](#) and [Karpenter](#) for node scaling, responding to built-in Amazon CloudWatch metrics, engine-specific performance indicators, or custom metrics defined by your organization. SageMaker HyperPod provides comprehensive observability over cluster resources and software components with [Amazon CloudWatch Container Insights](#), [Amazon Managed Service for Prometheus](#), and [Amazon Managed Grafana](#).

Inference engine choice

You can select from [Deep Learning Container](#) images that are optimized for various frameworks and maintained by AWS. Or, you can deploy custom container images with specialized inference engines, which supports diverse deployment scenarios.

Inference configuration

The platform provides comprehensive configuration flexibility for both model and system parameters. Organizations can do the following:

- Define precision levels.
- Implement prompt caching strategies.
- Configure model sharding across GPUs, Trainium NeuronCores, or even nodes.
- Select quantization approaches.
- Enable multi-node deployments.
- Choose communication protocols.
- Support various client types through the runtime and container configuration.

Supported clients and protocols

Deployed inference workloads can be accessed by any of the following approaches, providing maximum integration flexibility:

- Directly over Application Load Balancers over HTTPS
- Through the [Amazon SageMaker Runtime API](#)
- By using the AWS SDK

- By using the SageMaker SDK
- Through any protocol or client compatible with the chosen inference engine

Self-managed inference

Self-managed inference on Amazon EC2, Amazon EKS, or Amazon ECS provides organizations with complete control over their deployment environment and configuration.

Infrastructure management

Organizations maintain full control over infrastructure configuration and management. On Amazon EC2, teams manage instances directly. On Amazon EKS, Kubernetes provides container orchestration. On Amazon ECS, AWS manages the container orchestration layer while organizations control task and service definitions.

Training compute reuse

Customers can expand or reallocate their existing Amazon EC2 capacity—currently used for non-inference tasks like training or evaluation—to support AI inference workloads.

Pricing model

Customers are charged based on provisioned compute instances, storage, and networking resources. [On-Demand Capacity Reservations and Capacity Blocks for ML](#) provide commitment-based pricing options that deliver reduced rates and guaranteed capacity for planned workloads.

Model architecture support

The environment accommodates any model format or architecture, enabling deployment of models built with any framework, custom architectures, or proprietary implementations.

Automatic scaling

Scaling mechanisms are tailored to each AWS service's capabilities:

- **Amazon EKS** – [Kubernetes Horizontal Pod Autoscaler \(HPA\)](#) or [KEDA](#) adjust pod replicas based on resource utilization or custom metrics. Also, for Amazon EKS, [Karpenter](#) dynamically provisions and scales nodes to match workload demands efficiently.

- **Amazon ECS** – [Service autoscaling](#) adjusts task counts based on CloudWatch metrics, target tracking policies, or step scaling configurations.
- **Amazon EC2** – [Autoscaling groups](#) manage instance fleets using dynamic scaling policies, predictive scaling, or scheduled actions.

All approaches can respond to built-in metrics (such as CPU, memory, and network) and to engine-specific performance indicators (such as request latency, queue depth, and throughput). They can also respond to custom metrics tailored to specific application requirements from [Amazon CloudWatch](#), [Amazon Managed Service for Prometheus](#), or third-party sources.

Inference engine choice

You can select from [Deep Learning Container](#) images that are optimized for popular frameworks and maintained by AWS. Or, they can deploy custom container images with any inference engine or specialized software stack.

Inference configuration

The environment provides complete customization of system configuration. Organizations can implement any quantization level, configure model sharding strategies, enable prompt caching mechanisms, define batching behaviors, select communication protocols, and optimize for specific performance characteristics.

Supported clients and protocols

Inference API servers can be exposed through Application Load Balancers, Network Load Balancers, or Kubernetes ingress controllers. All approaches support any protocol or client compatible with the deployed inference engine or container. This flexibility enables integration with diverse application architectures and client requirements.

Model optimization techniques

You can apply several key optimization techniques to improve gen AI model performance and efficiency.

Pruning

Pruning removes less important or redundant connections and neurons from a model, resulting in a sparser, more efficient network. This process can involve removing entire units like neurons or channels or setting individual weight parameters to zero based on various importance criteria. The goal is to reduce the model's size and computational requirements while maintaining its performance capabilities. When properly implemented, pruning can significantly decrease the resource requirements of a model without substantially impacting its accuracy. Pruning is an effective optimization technique for deployment scenarios where efficiency is crucial.

Quantization

Quantization reduces both model and cache memory footprint while accelerating memory I/O and compute latency. This approach works by converting higher precision weights to lower precision formats, such as converting from FP16 to INT8. The key advantage of quantization is its ability to significantly reduce model size and improve performance while minimizing accuracy loss. Quantization is particularly effective for deployment in resource-constrained environments.

Model compilation

Model compilation optimizes the underlying operators for specific hardware architectures. This process involves combining multiple operations for more efficient execution and eliminating redundant computations throughout the model. By creating hardware-specific optimized code paths, compilation ensures that models run as efficiently as possible on the target hardware platform. Compilation can result in improved inference speed and resource utilization.

Speculative decoding

Speculative decoding is a technique to speed up the decoding process of large LLMs. It optimizes models for latency without compromising the quality of the generated text.

This technique can use a smaller but faster "draft" model to propose candidate tokens, which are then validated by the larger but slower "target" model. This is known as the draft-target approach. Alternatively, the [Extrapolation Algorithm for Greater Language-model Efficiency \(EAGLE\)](#) approach attaches a lightweight "EAGLE head" directly to the target model, rather than using a separate draft model. The EAGLE head generates an entire tree of potential token candidates by extrapolating from the target model's internal hidden state features. The target model can then efficiently verify and prune this draft token tree in a single parallel pass.

In either case, at each iteration, the draft or EAGLE mechanism generates multiple candidate tokens. The target model verifies the tokens, and if it finds that a particular token is not acceptable, it rejects that token and regenerates it. So, the target model both verifies tokens and generates a small amount of them.

The draft or EAGLE mechanism is significantly faster than the target model. It generates all the token candidates quickly and then sends them to the target model for parallel verification. This speeds up the final response compared to standard sequential token generation.

SageMaker AI offers a [pre-built draft model](#) that you can use, so you don't have to build your own. If you prefer to use your own custom draft model, SageMaker AI also supports this option.

Artifact storage

Efficient artifact storage maintains model weights and parameters in a ready-to-use state, enabling quick pre-loading and caching of examples. This optimization technique reduces cold-start times and streamlines the storage and retrieval of model weights. Proper artifact management is important for maintaining consistent performance in production environments, especially with large models, where checkpoints can approach hundreds of gigabytes. SageMaker AI provides [functionalities](#) to facilitate artifact management.

Applying inference optimizations in SageMaker AI

SageMaker AI leverages the techniques described earlier. With SageMaker AI, you can improve the performance of your generative AI models by applying inference optimization techniques. By optimizing your models, you can attain better cost performance for your use case. When you optimize a model, you choose which supported [optimization technique](#) to apply, including quantization, speculative decoding, compilation, and fast model loading. After your model is optimized, you can run an evaluation to see performance metrics for latency, throughput, and price.

Right-sizing and auto-scaling an inference system

Fundamental to designing an inference system is the selection of the underlying compute infrastructure and the policies for dynamically scaling it based on inference demand.

Selecting Amazon EC2 instance types and instance count

Choosing a suitable Amazon Elastic Compute Cloud (Amazon EC2) instance type is the first step in right-sizing infrastructure for a given model. EC2 instance types provide a flexible choice of GPUs, CPUs, and custom chips like AWS Inferentia and AWS Trainium. The instance type and count on a given instance constrains the throughput compute and memory that is available for hosting the model.

Model memory requirements should be satisfied by the available high-bandwidth memory on each individual instance (for example, using multiple GPUs through tensor parallelism). Model compute requirements can be split among multiple instances that serve multiple copies of the model. These instances are fronted by an inference API server that routes requests to individual instances. An example of a compute requirement is the required Requests per Second (RPS).

Inference memory requirements are mainly driven by model parameters and key-value (KV) cache size. You can estimate the memory requirements for model parameters based on the parameter count and the numerical precision of the weights. The following table shows approximate memory requirements for model parameters at a given precision for popular model sizes.

Model size (parameters)	16-bit	8-bit	4-bit
7B	14 GB	7 GB	3.5 GB
13B	26 GB	13 GB	6.5 GB
70B	140 GB	70 GB	35 GB

The memory requirements for KV cache scale roughly linearly with the number of tokens in the context length, the batch size (for example, multiple users), and the number of attention heads. They also depend on the numerical precision used to store the cache. If multiple users are using

the same endpoint on a given GPU, this will additionally scale the memory requirements. Several calculation tools can estimate the KV cache size for open source models, for example, [Hugging Face](#) and [lmcache.ai](#). As a rough rule of thumb, the KV cache can take up memory on the same order as the model weights themselves (often approximately 50% but can be higher for long contexts).

Most modern inference backends support tensor parallelism, which distributes the model parameters and KV cache across multiple GPUs on the same node. For example, a p5.48xlarge instance has 8× H100 GPUs, each with 80 GB of memory, for a total of 640 GB. This approach allows inference on models whose parameters and KV cache exceed the memory of a single GPU. The tradeoff is some synchronization overhead due to cross-GPU communication.

Inference compute requirements are mainly driven by the size and architecture of the model and the workload characteristics (for example, concurrent requests, acceptable latency, and sequence length). Primary indicators for the capability of an inference setup are Requests per Second (RPS) for throughput and Time to First Token (TTFT) for latency. Based on the use case requirements (for example, number of concurrent users), these indicators can be used to estimate the required number of GPUs and instances. Following are two main options to deduct the achievable throughput and latency based on a given model and use case:

- If the model is a (potentially fine-tuned) popular open source model, throughput benchmark results for popular GPU and instance types are available on websites such as [artificialanalysis.ai](#). Throughput benchmark results for several Llama, Mistral, and Falcon models are available in [Benchmark and optimize endpoint deployment in Amazon SageMaker JumpStart](#) (AWS Blog).
- If the model is heavily customized or the GPU choices are not included in public benchmarks, individual benchmarks can be performed using open source tools such as [LLMPerf](#) and the [vLLM](#) benchmark suite.

The throughput metric RPS depends heavily on the specific requests that were used within the benchmark. Popular benchmarking datasets such as [ShareGPT](#) are based on real user conversations and can approximate real-world workloads.

Although we generally recommend accelerated computing instance types (Inf, Trn, G and P [instance families](#)) for most inference use cases, CPU-based instances can be cost-effective for inference with smaller quantized models.

Configuring auto-scaling policies

After the baseline infrastructure setup is defined, auto-scaling ensures capacity adjusts dynamically with demand. This approach helps to ensure both sufficient capacity to meet latency and throughput requirements and effective resource utilization, minimizing idle infrastructure costs.

Gen AI inference systems typically use horizontal scaling (adding or removing instances) in contrast to vertical scaling (adjusting instance types or GPU utilization).

Scaling strategies can generally be classified as reactive (based on the current load on the system) and scheduled (for workloads with predictable temporal patterns). Both strategies can be combined, depending on the use case specifications.

Auto-scaling policies rely on metrics that reflect the load of the inference system. Common metrics include:

- **Requests per Second** - Tracking RPS against the maximum tested throughput per instance type (see the previous section) helps maintain predictable performance.
- **KV Cache Usage** - Average key-value cache utilization across instances indicates that GPUs are saturated and additional instances might be required.
- **Request Queue Length** or **Pending Requests** – A growing request queue indicates demand is exceeding available throughput. This metric correlates directly with user-facing latency.

AWS provides multiple approaches for implementing auto-scaling, depending on the chosen inference service:

- Amazon Bedrock provides fully managed and automatic scaling of the inference capacity for both publicly available foundation models and models imported through [Custom Model Import](#). No manual setup is required.
- Amazon SageMaker [endpoint auto scaling](#) provides fully managed scaling based on configurable thresholds on invocation metrics as well as custom CloudWatch metrics such as CPU utilization.
- [Amazon SageMaker HyperPod](#) and inference setups based on Amazon EKS can use Kubernetes-based auto scaling functionality through KEDA and Karpenter.
- [Amazon ECS Managed Instances](#) provides flexible cost-effective auto-scaling for inference workloads.

There are two primary challenges with auto-scaling setups:

- **Cold start latency** - Scale-out events have latency due to instance provisioning, container image pull, and model weight loading.
- **Capacity availability** - Depending on the AWS Region, the desired instance types can experience varying levels of availability, increasing latency in scale-out events or even prohibiting them during certain periods.

System-level optimizations

Beyond instance selection and scaling, the following system-level optimizations can improve inference efficiency, reduce latency, and increase throughput:

- **Routing and load balancing** - Efficient request routing ensures balanced utilization across GPUs and instances. Common strategies include:
 - **Round-robin routing** for even distribution of independent requests.
 - **Session-aware routing** to keep user conversations on the same instance, preserving KV cache locality.
 - **Latency-aware routing** to send requests to the least-loaded instance for reduced queueing delay.
- **Throughput and latency** - Batch size is one of the primary levers for tuning performance. Smaller batches favor low latency, while larger batches improve throughput. Optimal settings depend on workload characteristics (for example, interactive chat or batch inference).
- **Prefix caching** - Caching identical prompt prefixes avoids redundant compute and decoding, significantly lowering latency for repeated queries. This approach is especially effective in applications with high overlap in user prompts. Frameworks such as vLLM (PagedAttention), Text Generation Inference (TGI), SGLang, and Llama.cpp natively support prefix caching.
- **Chunked prefill** - Large context inputs can stall GPUs during prefill. Splitting prefill into smaller chunks reduces idle decoding cycles, improving utilization for long-sequence workloads.
- **Continuous batching** - Continuous batching dynamically groups incoming requests, prioritizing memory I/O and compute to maximize throughput while keeping latency within acceptable bounds. This approach prevents resource underutilization when request patterns are uneven.
- **Sequence size tuning** - Right-sizing the maximum sequence length reduces unnecessary computation for workloads that rarely need the full context window. This tuning balances memory footprint, throughput, and latency.

Implementing security and responsible AI

Building generative AI (gen AI) models involves complex technical and business decisions. To speed up experimentation, some [responsible AI \(RAI\)](#) evaluations might be overlooked initially. However, conducting an RAI review *before* moving to inference helps organizations strengthen risk assessment and resource management through a structured evaluation process. This pre-release review serves as the final controlled stage where organizations can thoroughly assess models for potential harm. For example, organizations can assess models for bias amplification, privacy violations, safety risks, or misalignment with organizational values.

The RAI review consists of two phases:

- Before moving to inference
- After inference deployment

Before moving to inference

To establish a foundation for responsible deployment, implement a model registry to track lineage, versions, and artifacts, ensuring traceability throughout the model's lifecycle. The model evaluation report provides a detailed assessment that quantifies robustness, fairness across demographic groups, and potential failure modes. This assessment offers visibility into unexpected behavior and supports informed trade-off discussions. These evaluations guide the creation of comprehensive governance structures, including formal approval workflows, stakeholder sign-offs, and escalation paths that maintain organizational awareness of potential risks and impacts.

To complement these governance mechanisms, implement technical safeguards such as multi-layered defensive controls for input validation, output filtering, and personally identifiable information (PII) sanitization. Finally, practical access controls—including API key authentication, identity-based authorization, and network isolation—protect inference endpoints from unauthorized use, abuse, and resource exhaustion.

Model assessment

During the model assessment phase, the evaluation report helps the organization understand the model's robustness and explainability through quantitative and qualitative analyses. This analysis includes evaluating accuracy metrics, fairness scores, and error distributions, complemented by human-in-the-loop reviews and A/B testing to validate real-world performance. The report should

also provide visibility into unexpected behavior, foster trade-off discussions, and guide the team in incorporating mechanisms for exception management.

Documentation preparation

With the model assessment report, the team can now develop internal documentation to record trade-offs and decisions made during the process. This documentation provides valuable context for future development iterations. When the documentation is revisited, it helps the team understand the limitations and requirements of the previous project timeframe.

A model card should include detailed specifications of the model architecture, training methodology, and hyperparameter configurations, along with performance metrics across diverse datasets and demographic groups. It must also explicitly outline both intended use cases and scenarios where the model should not be applied, establishing clear boundaries for appropriate deployment.

Technical safeguards

Technical safeguards protect the system from potential harm through engineering controls and architectural design choices. Before model deployment, developers must establish multi-layered defensive mechanisms, including the following:

- Input validation to detect and reject adversarial prompts
- Output filtering systems to identify and block harmful, misleading, or biased content
- Thorough sanitization processes for PII

[Amazon Bedrock Guardrails](#) provides safeguards that you can configure for your generative AI applications based on your use cases and responsible AI policies.

Governance setup

Before deploying a model, organizations should establish formal deployment approval workflows. These procedures help to ensure that all stakeholders review evaluation reports and have access to shared documentation such as model cards and decision records. This approach fosters organizational awareness of potential risks and impacts associated with deployment. A structured approval process should be completed before release. Governance execution must align with previously documented requirements and include technical controls with clear purposes, such as configuring access permissions and rate limits to prevent resource abuse.

It's essential to control access to inference endpoints to prevent abuse, unauthorized use, and excessive consumption—particularly for public- or customer-facing systems. The following complementary access control methods are commonly used:

- **API key authentication** – API keys provide a simple form of authentication that is natively supported by most popular frameworks and API specifications such as the [OpenAI API](#). Managed platforms such as Amazon Bedrock and open source frameworks such as [LiteLLM](#) provide options for issuing API keys to users through the UI and programmatically. API keys are associated with allowed models and usage limits. API keys should be securely loaded from an environment variable or key management service on the server and not be exposed to client-side code. Furthermore, API keys should only be valid short-term and rotated frequently, to prevent abuse in case of accidental leakage.
- **Identity-based authentication** – Identity-based authentication relies on short-lived credentials and signed requests instead of shared secrets like API keys. Within AWS, callers assume AWS Identity and Access Management (IAM) [roles](#). Authorization is enforced through IAM [policies](#) that scope allowed models, endpoints, and actions (for example, invoke, batch, and embeddings). In public- and customer-facing applications, end users typically authenticate through OAuth2/OIDC (for example, through [Amazon Cognito](#)). Then, identities are mapped to their respective IAM roles.
- **Network isolation** – For company-internal inference endpoints, network isolation can be enforced by only exposing access with internal load balancers within a [virtual private cloud \(VPC\)](#) and by scoping access with [security groups](#) and [network access control lists](#). If public ingress is unavoidable, use IP allowlists where possible. Also in that situation, the inference endpoint should be fronted by services such as [Amazon CloudFront](#) and [AWS WAF](#). These AWS services prevent distributed denial of service (DDoS) attacks and other common exploits. Network isolation complements API keys and identity-based authentication so that even leaked credentials can't reach an endpoint directly.

In addition to access control, request payloads should always be encrypted in transit by enforcing SSL/TLS.

After inference deployment

A gen AI model deployment guided by RAI practices integrates three key elements to ensure models remain safe, effective, and aligned with organizational values:

- Continuous monitoring provides observability through real-time tracking of performance metrics such as latency, throughput, and error rates. It also observes contextual metadata that enables analysis across dimensions like criticality, business domain, compliance, and resource use.
- When issues arise, incident response mechanisms act as a rapid intervention system. Predefined procedures and clear roles and responsibilities help stakeholders access information and resources quickly to minimize impact.
- Complementing these systems, feedback mechanisms capture user experiences that automated metrics might overlook. These mechanisms provide early warnings for hallucinations, bias, or harmful outputs while incorporating diverse perspectives to better reveal model limitations in real-world contexts.

Continuous monitoring

Continuous monitoring provides real-time visibility into model behavior and system performance across the inference lifecycle. Amazon SageMaker AI enables this observability through components such as [Model Monitor](#) for detecting data and concept drift, [SageMaker Clarify](#) for bias detection and explanations, and [SageMaker Debugger](#) for real-time performance tracking. These tools, combined with tagging model interactions by use case, generate contextual metadata that supports deeper analysis and operational insights. For example, critical versus non-critical workload tags enable prioritization during incident response. Business domain tags support performance comparisons. Compliance tags facilitate automated policy enforcement. Cost center tags improve resource tracking and financial accountability.

Incident response mechanisms

Establishing incident response mechanisms helps to mitigate potential harm and keep deployed gen AI models reliable. Address incidents as quickly as possible to minimize their impact on end users. These systems must also be designed for accessibility, so that response stakeholders and subject owners have immediate access to relevant information and resources.

Feedback collection mechanisms

The indeterministic nature of gen AI models can still lead to unexpected behavior for your customers. Customer feedback serves as an early warning system, identifying hallucinations, biases, or harmful responses that automated quality metrics may miss. Incorporating feedback from diverse user groups helps organizations better understand model limitations and real-world data distributions. By systematically collecting, categorizing, and analyzing both positive and

negative feedback, organizations can continuously refine their understanding of use cases, user values, and expectations.

AWS Partner Network

The [AWS Partner Network \(APN\)](#) is a global community of partners that leverage programs, expertise, and resources to build, market, and sell customer solutions. [AWS AI Competency Partners](#) drive the advancement of services, tools, data strategy, and infrastructure pivotal for implementing generative AI (gen AI) technologies across diverse industries. Their customer success and technical best practices have been validated by gen AI specialists at AWS. Partners in the Generative AI category can help organizations in the following ways:

- **Consulting services** – Partners with deep expertise can assist organizations with the following:
 - Adopt and strategize gen AI.
 - Build and test gen AI applications.
 - Train and customize foundation models.
 - Operate and maintain gen AI applications and models.
 - Protect gen AI workloads.
 - Define responsible AI principles and frameworks.
- **Gen AI applications** – Partners provide tools and solutions that deliver customer value across the full AWS gen AI stack, from foundation models to end user applications.
- **Foundation models and app development** – Partners can help organizations with foundation model training, customization, and the development of gen AI-powered applications.
- **Infrastructure and data** – Partners offer expertise in areas like data strategy, governance, storage, and the underlying infrastructure required to power gen AI workloads on AWS.

By leveraging the capabilities of AWS Generative AI Competency Partners, organizations can accelerate their generative AI initiatives, drive innovation, and transform their businesses.

Resources

AWS service documentation

- [Amazon Bedrock](#)
- [Amazon Bedrock Custom Model Import](#)
- [Amazon Bedrock Guardrails](#)
- [Amazon SageMaker AI](#)
- [Amazon Elastic Compute Cloud](#)
- [Amazon Elastic Container Service](#)
- [Amazon Elastic Kubernetes Service](#)
- [How inference works in Amazon Bedrock](#)
- [Deploy models for inference on Amazon SageMaker AI](#)
- [Deploying models on Amazon SageMaker HyperPod](#)

Other AWS resources

- [AI on Amazon EKS](#) (AWS GitHub repository)
- [Benchmark and optimize endpoint deployment in Amazon SageMaker JumpStart](#) (AWS Blog)
- [Deploy LLMs on Amazon EKS using vLLM Deep Learning Containers](#) (AWS Blog)
- [Managed Tiered KV Cache and Intelligent Routing for Amazon SageMaker HyperPod](#) (AWS Blog)
- [Responsible AI](#)
- [Amazon SageMaker HyperPod workshop](#) (AWS Workshop)

Other resources

- [EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Time Test](#) (arXiv)
- [An Introduction to Speculative Decoding for Reducing Latency in AI Inference](#) (Nvidia)
- [Overview of quantization techniques](#) (Hugging Face)

Contributors

The following individuals contributed to this guide.

Main authors:

- Oussama Kandakji, Senior AI/ML Go-To-Market Specialist Solutions Architect, AWS
- Nicolas Jourdan, Senior AI/ML Go-To-Market Specialist Solutions Architect, AWS

Co-authors:

- Mia Chang, AI/ML Go-To-Market Specialist Solutions Architect, AWS
- Marco Sgroi, Principal AI/ML Go-To-Market Specialist, AWS

Reviewers:

- Roy Allela, Senior AI/ML Specialist Solutions Architect, AWS
- Robert Northard, Principal Containers Specialist Solutions Architect, AWS

Document history

The following table describes significant changes to this guide.

Change	Description	Date
Initial publication	—	December 18, 2025